

## 1. Objectives

Familiar with Git commands like git init, git status, git add, git commit, git push, and git pull.

In this hands-on lab, you will learn how to

- Setup your machine with Git Configuration
- Integrate notepad++.exe to Git and make it a default editor
- Add a file to source code repository

## Prerequisites

- Install Git Bash client in your machine

Notes\*:

Please follow the below steps for creating a free account in GitHub.

Don't use cognizant credentials to login to GitHub.

Estimated time to complete this lab: **30 minutes**.

## Step 1: Setup your machine with Git Configuration

To create a new repository, signup with GitLab and register your credentials

Login to GitLab and create a "GitDemo" project

1. To check if Git client is installed properly: Open Git bash shell and execute

```
$ git version  
git version 2.21.0.windows.1
```

If output shows Git with its version information that indicates, that Git Client installs properly.

2. To configure user level configuration of user ID and email ID execute

```
$ git config --global user.name "username"  
$ git config --global user.email "username@cognizant.com"
```

3. To check if the configuration is properly set, execute the following command.

```
$ git config --global --list  
user.name=username  
user.email=username@cognizant.com
```

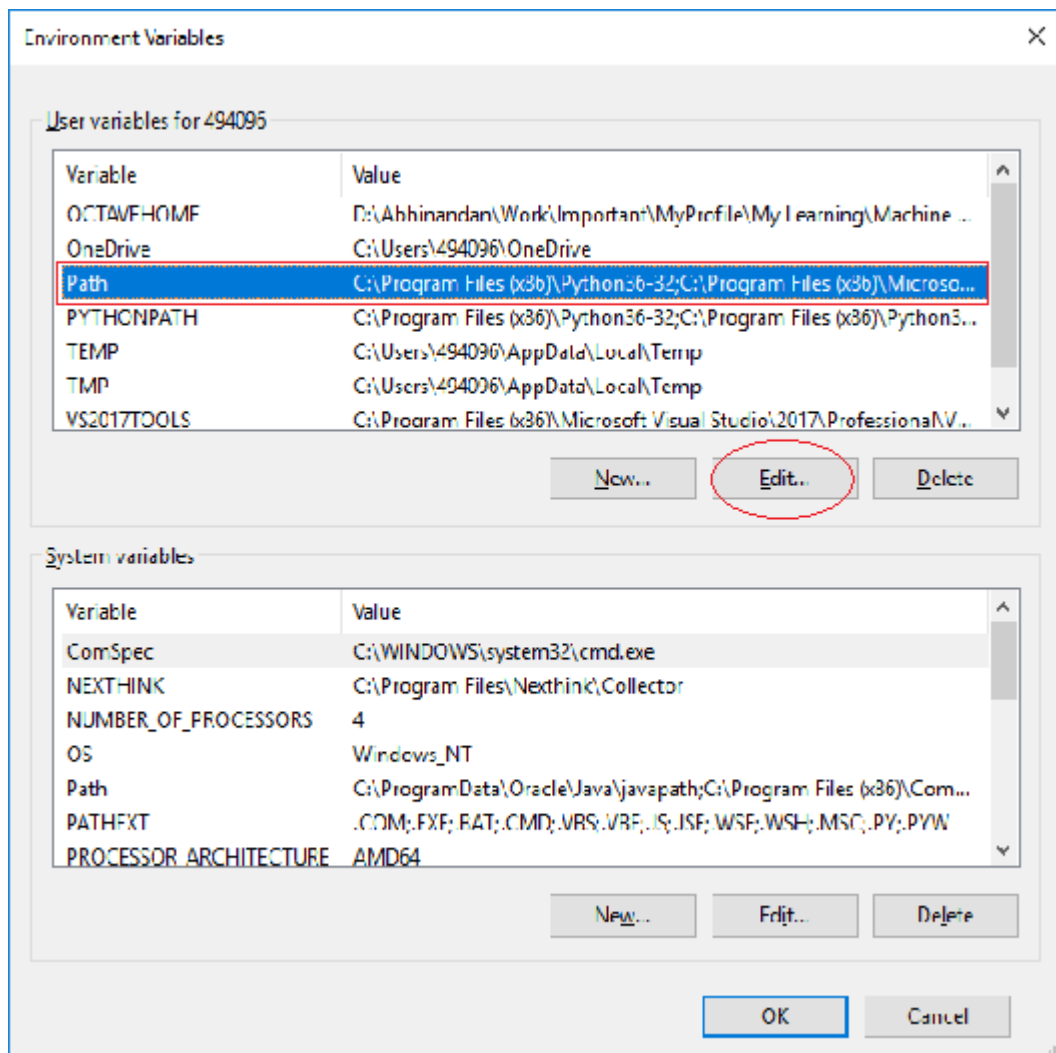
## Step 2: Integrate notepad++.exe to Git and make it a default editor

1. To check, if notepad++.exe execute from Git bash

```
$ notepad++  
bash: notepad++: command not found
```

If Git bash could not able to recognize notepad++ command that implies notepad++.exe is not added to the environment path variable.

To add path of notepad++.exe to environment variable, go to control panel -> System -> Advanced System settings. Go to Advanced tab -> Environment variables -> Add path of notepad++.exe to the path user variable by clicking on "Edit"



2. Exit Git bash shell, open bash shell and execute

```
$ notepad++
```

Now, notepad++ will open from Git bash shell

3. To create an alias command for notepad++.exe, execute

```
$ notepad++.exe bash -profile
```

It will open notepad++ from bash shell, and create a user profile by adding the line in notepad++

```
alias npp='notepad++.exe -multiInst -nosession'
```

4. To configure the editor, execute the command

```
$ git config --global core.editor "notepad++.exe -multiInst -nosession"
```

5. To verify if notepad++ is the default editor, execute the command

```
$ git config --global -e  
hint: Waiting for your editor to close the file... _
```

Here '-e' option implies editor

It will show the entire global configuration as shown below,

```
[user]  
  name = username  
  email = username@cognizant.com  
[core]  
  editor = notepad++.exe -multiInst -nosession
```

### Step 3: Add a file to source code repository

1. Open Git bash shell and create a new project "GitDemo" by executing the command

```
$ git init GitDemo  
Initialized empty Git repository in D:/Development_Avecto/GitDemo/.git/
```

2. Git bash initializes the "GitDemo" repository. To verify, execute the command

```
$ ls -al  
total 8  
drwxr-xr-x 1 1049089 0 Jan 13 11:54 ./  
drwxr-xr-x 1 1049089 0 Jan 13 11:54 ../  
drwxr-xr-x 1 1049089 0 Jan 13 11:54 .git/
```

It will display all the hidden files in the Git "working directory".

3. To create a file “**welcome.txt**” and add content to the file, execute the command

```
$ echo "Welcome to the version control" >> welcome.txt
```

4. To verify if the file “welcome.txt” is created, execute

```
$ ls -al
total 9
drwxr-xr-x 1 494096 1049089  0 Jan 13 12:02 ./
drwxr-xr-x 1 494096 1049089  0 Jan 13 11:54 ../
drwxr-xr-x 1 494096 1049089  0 Jan 13 12:01 .git/
-rw-r--r-- 1 494096 1049089 31 Jan 13 12:02 welcome.txt
```

5. To verify the content, execute the command

```
$ cat welcome.txt
Welcome to the version control
```

6. Check the status by executing

```
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)

        welcome.txt
```

Now the file “**welcome.txt**” is available in Git “working directory”

7. To make the file to be tracked by Git repository, execute the command

```
$ git add welcome.txt
warning: LF will be replaced by CRLF in welcome.txt.
The file will have its original line endings in your working directory
```

8. To add multi line comments, we are opening default editor to comment. Execute the command

```
$ git commit
```

Notepad++ editor will open and to add multi-line comment with default editor

9. To check if local and “Working Directory” git repository are same, execute git status

```
$ git status
On branch master
nothing to commit, working tree clean
```

**welcome.txt** is added to the local repository.

10. Signup with GitLab and create a remote repository “**GitDemo**”

11. To pull the remote repository, execute

git pull origin master

12. To push the local to remote repository, execute

git push origin master

Answer:

```
anju@Meghumi MINGW64 ~
$ git version
git version 2.50.1.windows.1

anju@Meghumi MINGW64 ~
$ git config --global user.name "username"

anju@Meghumi MINGW64 ~
$ git config --global user.email "username@cognizant.com"

anju@Meghumi MINGW64 ~
$ git config --global --list
user.name=username
user.email=username@cognizant.com

anju@Meghumi MINGW64 ~
$ notepad++
bash: notepad++: command not found
```

```
anju@Meghumi MINGW64 ~
$ notepad++

anju@Meghumi MINGW64 ~
$ notepad++.exe bash -profile
```

```
bash
1 alias npp='notepad++.exe -multiInst -nosession'
2
```

```
anju@Meghumi MINGW64 ~
$ git config --global core.editor "notepad++.exe -multiInst -nosession"

anju@Meghumi MINGW64 ~
$ git config --global -e
```

```
gitconfig
1 [user]
2   name = username
3   email = username@cognizant.com
4 [core]
5   editor = notepad++.exe -multiInst -nosession
6
```

```
anju@Meghumi MINGW64 ~
$ git init GitDemo
Initialized empty Git repository in C:/Users/anju/GitDemo/.git/

anju@Meghumi MINGW64 ~
$ cd GitDemo
```

```
anju@Meghumi MINGW64 ~/GitDemo (master)
$ ls -al
total 32
drwxr-xr-x 1 anju 197121 0 Aug 3 15:41 ./
drwxr-xr-x 1 anju 197121 0 Aug 3 15:36 ../
drwxr-xr-x 1 anju 197121 0 Aug 3 15:36 .git/
```

```
anju@Meghumi MINGW64 ~/GitDemo (master)
$ echo "Welcome to the version control" >> welcome.txt

anju@Meghumi MINGW64 ~/GitDemo (master)
$ ls -al
total 32
drwxr-xr-x 1 anju 197121 0 Aug 3 15:41 ./
drwxr-xr-x 1 anju 197121 0 Aug 3 15:36 ../
drwxr-xr-x 1 anju 197121 0 Aug 3 15:36 .git/
-rw-r--r-- 1 anju 197121 31 Aug 3 15:41 welcome.txt
```

```
anju@Meghumi MINGW64 ~/GitDemo (master)
$ cat welcome.txt
Welcome to the version control

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    welcome.txt

nothing added to commit but untracked files present (use "git add" to track)

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git add welcome.txt
warning: in the working copy of 'welcome.txt', LF will be replaced by CRLF the next time Git touches it
```

```
anju@Meghumi MINGW64 ~/GitDemo (master)
$ git commit
[master (root-commit) 0b449b5] Added welcome.txt This file is used for version control lab
1 file changed, 1 insertion(+)
create mode 100644 welcome.txt
```

```
anju@Meghumi MINGW64 ~/GitDemo (master)
$ git status
On branch master
nothing to commit, working tree clean

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git remote add origin https://github.com/anjanaswaminathan/GitDemo.git
```

```
anju@Meghumi MINGW64 ~/GitDemo (master)
$ git push origin master
info: please complete authentication in your browser...
Enumerating objects: 3, done.
Counting objects: 100% (3/3), done.
Writing objects: 100% (3/3), 280 bytes | 280.00 KiB/s, done.
Total 3 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/anjanaswaminathan/GitDemo.git
 * [new branch]      master -> master
```

## 2. Objectives

- Explain git ignore

- Explain how to ignore unwanted files using git ignore

In this hands-on lab, you will learn how to:

- Implement git ignore command to ignore unwanted files and folders

### Prerequisites

The following are the pre-requisites to complete this hands-on lab:

- Setting up Git environment
- Integrate notepad++ as a default editor
- A Git repository in the local system and a remote repository in GitLab

Notes\*:

Please follow the below steps for creating a free account in GitHub.

Do not use cognizant credentials to login to GitHub.

Estimated time to complete this lab: **20 minutes**.

Create a **“.log”** file and a **log folder** in the working directory of Git. Update the **.gitignore** file in such a way that on committing, these files (.log extensions and log folders) are ignored.

Verify if the git status reflects the same about working directory, local repository and git repository.

Answer:

### 1. Explain .gitignore

.gitignore is a file used in Git to tell the system **which files or folders it should ignore** (not track or commit).

This is helpful when you have files that are temporary, auto-generated, or not meant to be shared (like logs, caches, or configuration files).

When Git sees the .gitignore file, it follows the instructions inside it and skips tracking any files that match the patterns written there.

### 2. How to Ignore Unwanted Files Using .gitignore

To ignore unwanted files:

1. **Create a .gitignore file** in the root folder of your Git project.
2. **Write the names or patterns** of files or folders you want Git to ignore.



3. Git will automatically **skip those files** during add/commit operations.

Once added to .gitignore, Git won't show these files in git status, and they won't be committed to the repository.

```
anju@Meghumi MINGW64 ~
$ mkdir GitIgnoreDemo

anju@Meghumi MINGW64 ~
$ cd GitIgnoreDemo

anju@Meghumi MINGW64 ~/GitIgnoreDemo
$ git init
Initialized empty Git repository in C:/Users/anju/GitIgnoreDemo/.git/

anju@Meghumi MINGW64 ~/GitIgnoreDemo (master)
$ echo "Sample log data" > debug.log

anju@Meghumi MINGW64 ~/GitIgnoreDemo (master)
$ mkdir log

anju@Meghumi MINGW64 ~/GitIgnoreDemo (master)
$ echo "Another log entry" > log/app.log

anju@Meghumi MINGW64 ~/GitIgnoreDemo (master)
$ notepad++ .gitignore

anju@Meghumi MINGW64 ~/GitIgnoreDemo (master)
$ git status
On branch master

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        .gitignore

nothing added to commit but untracked files present (use "git add" to track)

anju@Meghumi MINGW64 ~/GitIgnoreDemo (master)
$ git add .gitignore
git commit -m "Add .gitignore to ignore .log files and log/ folder"
[master (root-commit) 89ded52] Add .gitignore to ignore .log files and log/ folder
1 file changed, 2 insertions(+)
create mode 100644 .gitignore

anju@Meghumi MINGW64 ~/GitIgnoreDemo (master)
$ git status
On branch master
nothing to commit, working tree clean
```

### 3. Objectives

- Explain branching and merging
- Explain about creating a branch request in GitLab
- Explain about creating a merge request in GitLab

In this hands-on lab, you will learn how to:

- Construct a branch, do some changes in the branch, and merge it with master (or trunk)

### Prerequisites

The following are the pre-requisites to complete this hands-on lab:

- Setting up Git environment with P4Merge tool for Windows

Notes\*:

Please follow the below steps for creating a free account in GitHub.

Do not use cognizant credentials to login to GitHub.

Estimated time to complete this lab: **30 minutes**.

Please follow the instruction to complete the hands-on. Each instruction expects a command for the Git Bash.

### **Branching:**

1. Create a new branch **"GitNewBranch"**.
2. List all the local and remote branches available in the current trunk. Observe the **"\*"** mark which denote the current pointing branch.
3. Switch to the newly created branch. Add some files to it with some contents.
4. Commit the changes to the branch.
5. Check the status with **"git status"** command.

### **Merging:**

1. Switch to the master
2. List out all the differences between trunk and branch. These provide the differences in command line interface.
3. List out all the visual differences between master and branch using **P4Merge tool**.
4. Merge the source branch to the trunk.
5. Observe the logging after merging using **"git log --oneline --graph --decorate"**
6. Delete the branch after merging with the trunk and observe the git status.

### **Answer:**

#### **1. Branching and Merging in Git**

##### **Branching**

- **Branching means creating a separate line of development from the main code (usually called master or main).**
- **You can work on new features or bug fixes without affecting the main codebase.**
- **Example: git branch feature-login creates a branch named feature-login.**

##### **Merging**

- **Merging means taking the changes from one branch (like feature-login) and combining them into another, usually into master.**
- **This helps bring new features or updates into the main project after testing.**
- **Command: git merge feature-login (while on master)**

## 2. Creating a Branch Request in GitLab

A Branch Request is not a technical term in GitLab, but it usually means creating a new branch in the GitLab interface.

Steps:

1. Go to your GitLab project.
2. Click on Repository → Branches.
3. Click New Branch.
4. Enter the name of the new branch (e.g., GitNewBranch).
5. Select the branch to base it on (e.g., master).
6. Click Create Branch.

Now you can push code changes to this branch using Git Bash.

## 3. Creating a Merge Request in GitLab

A Merge Request (MR) is a way to ask GitLab to review and merge code from one branch to another (e.g., from GitNewBranch into master).

Steps:

1. Go to your GitLab project.
2. Click Merge Requests → New Merge Request.
3. Select:
  - Source branch: The one with your changes (e.g., GitNewBranch)
  - Target branch: Usually master or main
4. Click Compare branches and continue.
5. Fill in a title and description for your MR.
6. Click Create Merge Request.
7. After code review or approval, click Merge to complete.

```
anju@Meghuni MINGW64 ~
$ cd GitDemo
anju@Meghuni MINGW64 ~/GitDemo (master)
$ git branch GitNewBranch
anju@Meghuni MINGW64 ~/GitDemo (master)
$ git branch -a
  GitNewBranch
* master
  remotes/origin/master
```

```

arju@Meghumi MINGW64 ~/GitDemo (master)
$ git checkout GitNewBranch
Switched to branch 'GitNewBranch'

arju@Meghumi MINGW64 ~/GitDemo (GitNewBranch)
$ echo "Branch File content" > branchFile.txt

arju@Meghumi MINGW64 ~/GitDemo (GitNewBranch)
$ git add branchFile.txt
warning: in the working copy of 'branchFile.txt', LF will be replaced by CRLF th
e next time Git touches it

arju@Meghumi MINGW64 ~/GitDemo (GitNewBranch)
$ git commit -m "Added branchFile.txt in GitNewBranch"
[GitNewBranch 755e282] Added branchFile.txt in GitNewBranch
1 file changed, 1 insertion(+)
create mode 100644 branchFile.txt

arju@Meghumi MINGW64 ~/GitDemo (GitNewBranch)
$ git status
On branch GitNewBranch
nothing to commit, working tree clean

arju@Meghumi MINGW64 ~/GitDemo (GitNewBranch)
$ git checkout master
Switched to branch 'master'

```

```

arju@Meghumi MINGW64 ~/GitDemo (master)
Viewing 1/1: 'branchFile.txt'
Launch 'p4merge' [Y/n]?
Viewing 1/1: 'branchFile.txt'
Launch 'p4merge' [Y/n]? Y
/mingw64/libexec/git-core/git-mergetool--lib: line 172: C:/Program Files/Perforce/p4merge.exe: No such file or directory
fatal: external diff died, stopping at branchFile.txt
$ git merge GitNewBranch
Updating 0b449b5..755e282
Fast-forward
 branchFile.txt | 1 +
1 file changed, 1 insertion(+)
create mode 100644 branchFile.txt

arju@Meghumi MINGW64 ~/GitDemo (master)
$ git log --oneline --graph --decorate
* 755e282 (HEAD -> master, GitNewBranch) Added branchFile.txt in GitNewBranch
- 0b449b5 (origin/master) Added welcome.txt This file is used for version contro
1 lab

arju@Meghumi MINGW64 ~/GitDemo (master)
$ git branch -d GitNewBranch
Deleted branch GitNewBranch (was 755e282).

arju@Meghumi MINGW64 ~/GitDemo (master)
$ git status
On branch master
nothing to commit, working tree clean

```

#### 4. Objectives

- Explain how to resolve the conflict during merge.

In this hands-on lab, you will learn how to:

- Implement conflict resolution when multiple users are updating the trunk (or master) in such a way that it results into a conflict with the branch's modification.

#### Prerequisites

The following are the pre-requisites to complete this hands-on lab:

- Hands-on ID: **"Git-T03-HOL\_001"**

Notes\*:

Please follow the below steps for creating a free account in GitHub.

Do not use cognizant credentials to login to GitHub.

Estimated time to complete this lab: **30 minutes**.

Please follow the instructions to complete the hands-on. Each instruction expect a command for the Git Bash.

1. Verify if master is in clean state.
2. Create a branch **"GitWork"**. Add a file "hello.xml".
3. Update the content of "hello.xml" and observe the status
4. Commit the changes to reflect in the branch
5. Switch to master.
6. Add a file **"hello.xml"** to the master and add some different content than previous.
7. Commit the changes to the master
8. Observe the log by executing **"git log --online --graph --decorate --all"**
9. Check the differences with Git diff tool
10. For better visualization, use P4Merge tool to list out all the differences between master and branch
11. Merge the bran to the master
12. Observe the git mark up.
13. Use 3-way merge tool to resolve the conflict
14. Commit the changes to the master, once done with conflict
15. Observe the git status and add backup file to the .gitignore file.
16. Commit the changes to the .gitignore
17. List out all the available branches
18. Delete the branch, which merge to master.
19. Observe the log by executing **"git log --online --graph --decorate"**

Answer:

### **What is a Merge Conflict?**

A **merge conflict** happens when Git is unable to automatically combine changes from two branches. This usually occurs when two people (or branches) change the same part of a file in different ways.

---

### **How to Resolve a Merge Conflict :**

1. **Start the merge process:** You attempt to combine two branches. Git will try to merge them automatically.

2. **Conflict is detected:** If Git cannot decide which change to keep, it marks the file as conflicted and pauses the merge.
3. **Identify the conflicting files:** You check which files have conflicts. Git clearly shows you which files need attention.
4. **Open the file with conflict:** Inside the file, Git marks the conflicting sections using special markers. These markers show both versions of the conflicting code.
5. **Manually resolve the conflict:** You read the conflicting changes and decide what the final version should be. You can choose one version, or combine both.
6. **Remove the conflict markers:** After choosing the correct version, you remove Git's conflict indicators from the file to clean it up.
7. **Mark the conflict as resolved:** Once the conflict is resolved, you let Git know that you have handled the problem.
8. **Complete the merge process:** Finally, you save your changes and complete the merge. The branches are now successfully merged.

```
anjul@Meghumi MINGW64 ~
$ cd GitDemo

anjul@Meghumi MINGW64 ~/GitDemo (master)
$ git status
On branch master
nothing to commit, working tree clean

anjul@Meghumi MINGW64 ~/GitDemo (master)
$ git checkout master
Already on 'master'

anjul@Meghumi MINGW64 ~/GitDemo (master)
$ git status
On branch master
nothing to commit, working tree clean

anjul@Meghumi MINGW64 ~/GitDemo (master)
$ git checkout -b GitWork
Switched to a new branch 'GitWork'

anjul@Meghumi MINGW64 ~/GitDemo (GitWork)
$ echo "<msg>Hello from GitWork branch</msg>" > hello.xml

anjul@Meghumi MINGW64 ~/GitDemo (GitWork)
$ echo "<msg>Updated from GitWork branch</msg>" > hello.xml

anjul@Meghumi MINGW64 ~/GitDemo (GitWork)
$ git status
On branch GitWork
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   hello.xml

no changes added to commit (use "git add" and/or "git commit -a")

anjul@Meghumi MINGW64 ~/GitDemo (GitWork)
$ git add hello.xml
warning: in the working copy of 'hello.xml', LF will be replaced by CRLF the next time Git touches it

anjul@Meghumi MINGW64 ~/GitDemo (GitWork)
$ git commit -m "Added hello.xml in GitWork branch"
[GitWork d5d3f4c] Added hello.xml in GitWork branch
1 file changed, 1 insertion(+), 1 deletion(-)
```

```

anju@Meghumi MINGW64 ~/GitDemo (GitWork)
$ git checkout master
Switched to branch 'master'

anju@Meghumi MINGW64 ~/GitDemo (master)
$ echo <msg>Hello from master branch</msg> > hello.xml

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git add hello.xml
warning: in the working copy of 'hello.xml', LF will be replaced by CRLF the next time Git touches it

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git commit -m "Added hello.xml in master branch"
On branch master
nothing to commit, working tree clean

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git log --oneline --graph --decorate --all
* d5d3f4c (GitWork) Added hello.xml in GitWork branch
* 8a43dd9 (HEAD -> master) Add backup files to .gitignore
* 965fac0 Master branch: added hello.xml
* d58e59 GitWork branch: updated hello.xml

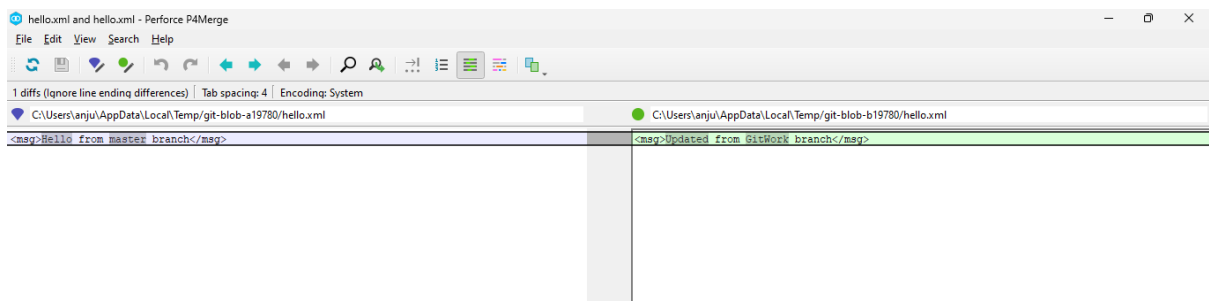
anju@Meghumi MINGW64 ~/GitDemo (master)
$ git difftool master GitWork
Viewing (1/1): 'hello.xml'
Launch 'p4merge' [Y/n]? Y

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git config --global merge.tool p4merge

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git config --global mergetool.p4merge.cmd '"C:/Program Files/Perforce/p4merge.exe" "$BASE" "$LOCAL" "$REMOTE" "$MERGED"'

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git difftool master GitWork
Viewing (1/1): 'hello.xml'
Launch 'p4merge' [Y/n]? Y

```



```

MINGW64/c/Users/anjul/GitDemo

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git merge GitWork
Updating 8a43dd9..d5d3f4c
Fast-forward
 hello.xml | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)

anju@Meghumi MINGW64 ~/GitDemo (master)
$ cat hello.xml
<msg>Updated from GitWork branch</msg>

anju@Meghumi MINGW64 ~/GitDemo (master)
$ cat hello.xml
<msg>Updated from GitWork branch</msg>

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git mergetool
No files need merging

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git add hello.xml

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git commit -m "Resolved merge conflict in hello.xml"
On branch master
nothing to commit, working tree clean

anju@Meghumi MINGW64 ~/GitDemo (master)
$ echo "*.orig" >> .gitignore

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git add .gitignore
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git touches it

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git commit -m "Ignore backup merge files"
[master 2f77250] Ignore backup merge files
 1 file changed, 1 insertion(+)

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git branch
GitWork
* master

```

```

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git branch
GitWork
* master

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git branch -d GitWork
Deleted branch GitWork (was d5d3f4c).

anju@Meghumi MINGW64 ~/GitDemo (master)
$ git log --oneline --graph --decorate
* 2f77250 (HEAD -> master) Ignore backup merge files
* d5d3f4c Added hello.xml in GitWork branch
* 8a43dd9 Add backup files to .gitignore
* 965fac0 Master branch: added hello.xml
* d58e59 GitWork branch: updated hello.xml

```

## 5. Objectives

- Explain how to clean up and push back to remote Git

In this hands-on lab, you will learn how to:

- Execute steps involving clean up and push back to remote Git.

## Prerequisites

The following are the pre-requisites to complete this hands-on lab:

- Hands-on ID: **“Git-T03-HOL\_002”**

Notes\*:

Please follow the below steps for creating a free account in GitHub.

Do not use cognizant credentials to login to GitHub.

Estimated time to complete this lab: **10 minutes.**

Please follow the instructions to complete the hands-on. Each instruction expects a command for the Git Bash.

20. Verify if master is in clean state.
21. List out all the available branches.
22. Pull the remote git repository to the master
23. Push the changes, which are pending from **“Git-T03-HOL\_002”** to the remote repository.
24. Observe if the changes are reflected in the remote repository.

Answer:

## How to clean up and push back to remote Git

### 1. Verify clean working state

- Run git status to check if there are any uncommitted changes.
- If there are changes, either commit them using git add . and git commit -m "message", or discard them.
- This ensures your local branch is “clean” before pushing.

### 2. Switch to the correct branch



- Use `git checkout <branch-name>` to move to the branch you want to update (e.g., main or your feature branch).

### 3. Pull latest changes from remote

- Run `git pull origin <branch-name>` to sync your local branch with the latest version from the remote repository.
- This helps avoid conflicts when pushing.

### 4. Push your branch to remote

- If your branch has new commits, run:  
`git push origin <branch-name>`
- This sends your local commits to the remote repository (GitHub, GitLab, etc.).

### 5. Verify on remote

- Open your repository in GitHub/GitLab.
- Check that your commits and files are visible on the correct branch

```
anju@Meghumi MINGW64 ~
$ cd 6410978

anju@Meghumi MINGW64 ~/6410978 (Git-T03-HOL_002)
$ git branch -a
* Git-T03-HOL_002
  main
  remotes/origin/Git-T03-HOL_002
  remotes/origin/HEAD -> origin/main
  remotes/origin/main

anju@Meghumi MINGW64 ~/6410978 (Git-T03-HOL_002)
$ git status
On branch Git-T03-HOL_002
nothing to commit, working tree clean

anju@Meghumi MINGW64 ~/6410978 (Git-T03-HOL_002)
$ git checkout main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
(use 'git push' to publish your local commits)

anju@Meghumi MINGW64 ~/6410978 (main)
$ git status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
(use 'git push' to publish your local commits)

nothing to commit, working tree clean

anju@Meghumi MINGW64 ~/6410978 (main)
$ git add .

anju@Meghumi MINGW64 ~/6410978 (main)
$ git commit -m "Clean up main branch"
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
(use 'git push' to publish your local commits)

nothing to commit, working tree clean

anju@Meghumi MINGW64 ~/6410978 (main)
$ git branch -a
* main
  remotes/origin/Git-T03-HOL_002
  remotes/origin/HEAD -> origin/main
  remotes/origin/main
```

```
anju@Meghumi MINGW64 ~/6410978 (main)
$ git branch -a
  Git-T03-HOL_002
* main
  remotes/origin/Git-T03-HOL_002
  remotes/origin/HEAD -> origin/main
  remotes/origin/main

anju@Meghumi MINGW64 ~/6410978 (main)
$ git pull origin main
From https://github.com/anjanaswaminathan/6410978
  branch      main       -> FETCH_HEAD
Already up to date.

anju@Meghumi MINGW64 ~/6410978 (main)
$ git checkout Git-T03-HOL_002
Switched to branch 'Git-T03-HOL_002'

anju@Meghumi MINGW64 ~/6410978 (Git-T03-HOL_002)
$ git push origin Git-T03-HOL_002
Everything up-to-date

anju@Meghumi MINGW64 ~/6410978 (Git-T03-HOL_002)
$ git checkout main
Switched to branch 'main'
Your branch is ahead of 'origin/main' by 1 commit.
(use "git push" to publish your local commits)

anju@Meghumi MINGW64 ~/6410978 (main)
$ git merge Git-T03-HOL_002
Already up to date.

anju@Meghumi MINGW64 ~/6410978 (main)
$ git push origin main
Total 0 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
To https://github.com/anjanaswaminathan/6410978.git
   3b55e4c..18e417e  main -> main
```